

InLegalBERT

Working

Steps:

1. Tokenization → Token IDs
2. Embedding Lookup
3. Transformer Layers
4. Mean Pooling for Sentence Embedding

Example:- The accused was granted bail.

1. Tokenization → Token IDs

```
tokens = ['[CLS]', 'the', 'accused', 'was', 'granted', 'bail', '.', '[SEP]']
```

Each token is assigned an ID from the vocabulary:

```
token_ids = [101, 2001, 3290, 2001, 3930, 4906, 1012, 102]
```

Example:- The accused was granted bail

2. Embedding Lookup

Each token_id maps to a token embedding vector of size 768 .

```
embeddings_layer_0 = [  
    [0.1, 0.2, 0.0, 0.3, ...],      # [CLS]  
    [0.0, 0.5, -0.1, 0.2, ...],     # the  
    [-0.2, 0.3, 0.4, -0.1, ...],    # accused  
    [0.1, 0.0, -0.2, 0.1, ...],     # was  
    [0.4, -0.3, 0.2, 0.0, ...],     # granted  
    [-0.1, 0.2, 0.1, 0.3, ...],    # bail  
    [0.0, -0.1, 0.0, 0.1, ...],     # .  
    [0.1, 0.0, 0.0, 0.2, ...],     # [SEP]  
]
```

This gives a matrix of shape:
(8 tokens, 768 dimensions)
→ 8×768

Example:- The accused was granted bail

3. Transformer Layers

Each of the 12 layers applies:

- Multi-head attention
- Feed-forward
- Layer norm + skip connections

Each token embedding is updated based on all others.

Example:- The accused was granted bail

3. Transformer Layers

- Multi-head attention

Each token attends to every other token in the sentence, including itself.

Example: In “The accused was granted bail.”

The word "bail" can attend to "accused" to understand who received bail.

Example:- The accused was granted bail

3. Transformer Layers

- Multi-head attention 

Captures contextual relationships between all tokens in parallel, not just nearby ones like RNNs.

What happens: For each token, compute:

Query vector (Q)

Key vector (K)

Value vector (V)

Compute attention weights: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$

Example:- The accused was granted bail

3. Transformer Layers

- Feed-Forward

Adds non-linearity and further transformation of attention output.

After attention, each token's updated vector is passed through a small MLP:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Operates independently on each token (position-wise).

Example:- The accused was granted bail

3. Transformer Layers

- Layer norm + skip connections $\longrightarrow$ Stabilizes training and preserve input signals:

Residual (skip) connection: $\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$

Adds original input x back after each sub-layer (attention and FFN).

Helps gradients flow better during training.

LayerNorm:

Normalizes activations to keep outputs numerically stable.

Example:- The accused was granted bail

3. Transformer Layers

After 12 layers, each token has a contextualized vector, capturing:

- Its meaning
- Its legal relevance
- Its relationship with other tokens (e.g., "accused" → "bail")

Example:- The accused was granted bail

3. Transformer Layers

Say after One attention layer 1, embeddings change to:

```
embeddings_layer_1 = [  
    [0.12, 0.18, 0.02, 0.31, ...],      # [CLS]  
    [0.02, 0.51, -0.08, 0.22, ...],     # the  
    [-0.18, 0.35, 0.42, -0.11, ...],    # accused  
    [0.13, -0.01, -0.19, 0.12, ...],     # was  
    [0.39, -0.28, 0.25, -0.01, ...],    # granted  
    [-0.09, 0.22, 0.11, 0.32, ...],     # bail  
    [0.01, -0.09, 0.01, 0.09, ...],     # .  
    [0.11, -0.01, 0.01, 0.21, ...],     # [SEP]  
]
```

Example:- The accused was granted bail

3. Transformer Layers

After 12 layers, these embeddings become fully contextualized

```
embeddings_final = [  
    [0.14, 0.17, 0.01, 0.33, ...],      # [CLS]  
    [0.05, 0.53, -0.09, 0.25, ...],     # the  
    [-0.15, 0.37, 0.45, -0.12, ...],    # accused  
    [0.12, -0.02, -0.21, 0.13, ...],    # was  
    [0.41, -0.25, 0.27, -0.03, ...],    # granted  
    [-0.08, 0.23, 0.13, 0.33, ...],     # bail  
    [0.02, -0.08, 0.02, 0.11, ...],     # .  
    [0.13, -0.02, 0.02, 0.23, ...],     # [SEP]  
]
```

Example:- The accused was granted bail

4. Mean Pooling for Sentence Embedding

Now we average across all token vectors:

$$\text{sum_0} = (0.14 + 0.05 - 0.15 + 0.12 + 0.41 - 0.08 + 0.02 + 0.13) = 0.64$$

$$\text{sum_1} = (0.17 + 0.53 + 0.37 - 0.02 - 0.25 + 0.23 - 0.08 - 0.02) = 0.93$$

$$\text{sum_2} = (0.01 - 0.09 + 0.45 - 0.21 + 0.27 + 0.13 + 0.02 + 0.02) = 0.60$$

$$\text{sum_3} = (0.33 + 0.25 - 0.12 + 0.13 - 0.03 + 0.33 + 0.11 + 0.23) = 1.23$$

...

$$\text{sum_767} = (0.33 + 0.25 - 0.12 + 0.13 - 0.03 + 0.33 + 0.11 + 0.23) = 1.23$$

Example:- The accused was granted bail

4. Mean Pooling for Sentence Embedding

Divide by 8:

```
sentence_embedding = [  
    0.64 / 8, # = 0.08  
    0.93 / 8, # = 0.11625  
    0.60 / 8, # = 0.075  
    1.23 / 8, # = 0.15375  
    ...  
    1.23/8, # = 0.15375  
]
```

Example:- The accused was granted bail

Final Output

Final Sentence Embedding (simulated, size=768):

[0.08, 0.11625, 0.075, 0.15375, ... 0.15375]